Assignment 3

BAN 130

Omar Al-Trad

Kevin Lu - 117926253

November 11th, 2025

In [57]:
```
import pandas as pd
# import pandas as pd first required before doing analysis
import numpy as np #import numpy as np in order to convert from character to numeri
```

Problem 1: Using the values for Day, Month, and Year in the raw data below, create a data frame containing a date based on these values (call it Date) and format this value using the MMDDYY10. format. Here are the Day, Month, and Year values:

25 12 2005

1 1 1960

21 10 1946

In [60]:
```
# Creating a list of Day, Months, and Years as provided
# For 25 12 2005 -> Days = 25, Month = 12, Year = 2005
# Want to format to MMDDYY10 to "12/23/2005"
Day=[25, 1, 21]
Month=[12, 1, 10]
Year=[2005, 1960, 1946]
print(Day, Month, Year) # Printing all three lists
```

```
[25, 1, 21] [12, 1, 10] [2005, 1960, 1946]
```

In [62]:
```
# Creating a dictonaries
datedata = {'Day':Day,
       'Month':Month,
       'Year':Year}
print(datedata)
```

```
{'Day': [25, 1, 21], 'Month': [12, 1, 10], 'Year': [2005, 1960, 1946]}
```

In [64]:
```
df = pd.DataFrame(datedata) # Creating a dataframe
print(df) #printing the original data before format using MMDDYY10
```

```
   Day  Month  Year
0   25     12  2005
1    1      1  1960
2   21     10  1946
```

In [66]:
```
df = pd.DataFrame(datedata)
df['Date'] = pd.to_datetime(df[['Day', 'Month', 'Year']]) #Format the date into yyy
Date1 = df[['Date']]
print(Date1) # Printing the standard date using yyyy-mm-dd format
```

```
        Date
0 2005-12-25
1 1960-01-01
2 1946-10-21
```

In [68]:
```
df['Date'] = df['Date'].dt.strftime('%m/%d/%Y') # Format this date value using MMDD
Update_Date = df[['Date']] #Going to show just the datw with format value using MMD
print(Update_Date) # Printing the results
#result should be like MM/DD/YYYY based on using MMDDYY10
```

```
         Date
0  12/25/2005
1  01/01/1960
2  10/21/1946
```

Problem 2: The below data contains character variables Age and Weight and numeric variables SS and Zip. Create a new data frame called Convert with new variables NumAge and NumWeight that are numeric values of Age and Weight, respectively, and CharSS and CharZip that are character variables created from SS and Zip. CharSS should contain leading 0s and dashes in the appropriate places for Social Security numbers and CharZip should contain leading 0s.

Age Weight SS Zip

23 155 132423222 8822

56 220 123457777 90210

74 95 12003004 78010

In [75]:
```
# Creating the list as usual
Age = ['23', '56', '74'] #Character variables
Weight = ['155', '220', '95'] #Character variables
SS = [132423222, 123457777, 12003004] #Numeric variables
Zip = [8822, 90210, 78010] #Numeric variables
print(Age, Weight, SS, Zip) #Printing all four lists as provided
```

```
['23', '56', '74'] ['155', '220', '95'] [132423222, 123457777, 12003004] [8822, 9021
0, 78010]
```

In [77]:
```
#Creating a dictanry
dict_data = {'Age':Age,
             'Weight':Weight,
             'SS':SS,
             'Zip':Zip}
print(dict_data)
```

```
{'Age': ['23', '56', '74'], 'Weight': ['155', '220', '95'], 'SS': [132423222, 123457
777, 12003004], 'Zip': [8822, 90210, 78010]}
```

In [79]:
```
df = pd.DataFrame(dict_data) #Creating a data frame
print(df) #printing an original data
```

```
  Age Weight         SS    Zip
0  23    155  132423222   8822
1  56    220  123457777  90210
2  74     95   12003004  78010
```

In [81]:
```
#Convert with new variables from character to numeric
df['NumAge'] = pd.to_numeric(df['Age'], errors='coerce') #NumAge -> Convert from ch
df['NumWeight'] = pd.to_numeric(df['Weight'], errors='coerce') #NumWeight -> Conver

#Convert with new variables from numeric to character
#CharSS -> Convert from numerical to character with leading 0s and dashes in the ap
df['CharSS'] = df['SS'].astype(str).str.zfill(9).str.replace(r'(\d{3})(\d{2})(\d{4}
df['CharZip'] = df['Zip'].astype(str).str.zfill(5) #CharZip -> Convert from numeric

#Making an updated datasets after conversion with new variables
Updated_data = df[['NumAge', 'NumWeight', 'CharSS', 'CharZip']]
print(Updated_data) # Printing the updated datasets after conversion with new varia
```

```
   NumAge  NumWeight       CharSS CharZip
0      23        155  132-42-3222   08822
1      56        220  123-45-7777   90210
2      74         95  012-00-3004   78010
```

Problem 3: Using the below stock data (containing variables Date and Price), compute daily changes in the prices and plot the result.

Stock Day 1 Day 2

AMGN 67.66 100

DELL 24.60 200

GE 34.50 100

HPQ 32.32 120

IBM 82.25 50

MOT 30.24 100

In [84]:
```
#Creating a list as usual
Stock = ['AMGN', 'DELL', 'GE', 'HPQ', 'IBM', 'MOT']
Day_1 = [67.66, 24.60, 34.50, 32.32, 82.25, 30.24]
Day_2 = [100, 200, 100, 120, 50, 100]
print(Stock, Day_1, Day_2) #printing from the list
```

```
['AMGN', 'DELL', 'GE', 'HPQ', 'IBM', 'MOT'] [67.66, 24.6, 34.5, 32.32, 82.25, 30.24]
[100, 200, 100, 120, 50, 100]
```

In [86]:
```
#Creating a dictanary
Dictanary_data = {'Stock':Stock,
                  'Day_1':Day_1,
                  'Day_2':Day_2}
print(Dictanary_data) #printing from dictanary
```

```
{'Stock': ['AMGN', 'DELL', 'GE', 'HPQ', 'IBM', 'MOT'], 'Day_1': [67.66, 24.6, 34.5,
32.32, 82.25, 30.24], 'Day_2': [100, 200, 100, 120, 50, 100]}
```

In [88]:
```
#Creating a data frame
df = pd.DataFrame(Dictanary_data)
print(df) #Printing the table of orginal datasets
```

```
  Stock  Day_1  Day_2
0  AMGN  67.66    100
1  DELL  24.60    200
2    GE  34.50    100
3   HPQ  32.32    120
4   IBM  82.25     50
5   MOT  30.24    100
```

In [90]:
```
df['Daily_Changes'] = df['Day_1'] - df['Day_2'] #Compute daily changes in the price
```

In [92]:
```
print(df) #Printing the updated table with an additional daily changes in the price
```

```
  Stock  Day_1  Day_2  Daily_Changes
0  AMGN  67.66    100         -32.34
1  DELL  24.60    200        -175.40
2    GE  34.50    100         -65.50
3   HPQ  32.32    120         -87.68
4   IBM  82.25     50          32.25
5   MOT  30.24    100         -69.76
```

In [94]:
```
#Making a table of just a stock with daily changes in the prices
Daily_changes_Table = df[['Stock', 'Daily_Changes']]
print(Daily_changes_Table)
```

```
  Stock  Daily_Changes
0  AMGN         -32.34
1  DELL        -175.40
2    GE         -65.50
3   HPQ         -87.68
4   IBM          32.25
5   MOT         -69.76
```

Problem 4: Data set Errors contains character variables Subj (3 bytes) and PartNumber (8 bytes). (See the partial listing here.) Create a temporary data frame (Check1) with any observation in Errors that violates either of the following two rules: first, Subj should contain only digits, and second, PartNumber should contain only the uppercase letters L and S and digits.

Display only the rows in Error.

Here is a partial listing of Errors: Subj Number Name

001 L1232 Nichole Brown

0a2 L887X Fred Beans

003 12321 Alfred 2 Nice

004 abcde Mary Bumpers

X89 8888S Gill Sandford

In [97]:
```
# Creating a list:
Subj = ['001', '0a2', '003', '004', 'X89']
Number = ['L1232', 'L887X', '12321', 'abcde', '8888S']
Name = ['Nicole Brown', 'Fred Beans', 'Alfred 2 Nice', 'Mary Bumpers', 'Gill Sandfo
print(Subj, Number, Name) #printing the list
```

```
['001', '0a2', '003', '004', 'X89'] ['L1232', 'L887X', '12321', 'abcde', '8888S']
['Nicole Brown', 'Fred Beans', 'Alfred 2 Nice', 'Mary Bumpers', 'Gill Sandford']
```

In [99]:
```
#Creating a dictanry
Error = {'Subj':Subj,
         'Number':Number,
         'Name':Name}
print(Error) #printing the dictanry
```

```
{'Subj': ['001', '0a2', '003', '004', 'X89'], 'Number': ['L1232', 'L887X', '12321',
'abcde', '8888S'], 'Name': ['Nicole Brown', 'Fred Beans', 'Alfred 2 Nice', 'Mary Bum
pers', 'Gill Sandford']}
```

In [101…
```
Error_data = pd.DataFrame(Error) #Creating a data frame
print(Error_data) #printing the original data frame
```

```
  Subj Number           Name
0  001  L1232   Nicole Brown
1  0a2  L887X     Fred Beans
2  003  12321  Alfred 2 Nice
3  004  abcde   Mary Bumpers
4  X89  8888S  Gill Sandford
```

In [103…

```
#Valid the errors for Subj and PartNumber (Number)
checking_for_subj = Error_data['Subj'].str.match(r'^\d+$') #Subj should only digits
checking_for_number = Error_data['Number'].str.match(r'^[L$0-9]+$') #PartNumber sho

Check= Error_data[-(checking_for_subj & checking_for_number)] #Making the datasets
print(Check) #printing the only rows in error
```

```
  Subj Number           Name
1  0a2  L887X     Fred Beans
3  004  abcde   Mary Bumpers
4  X89  8888S  Gill Sandford
```

Problem 5: You invest 1,000 a year at 4.25% interest, compounded quarterly. How many years will it take to reach $30,000? Do not use compound interest formulas. Rather, use “brute force” methods with DO WHILE or DO UNTIL statements to solve this problem.

In [108…

```
invest = 1000
year_interest_rate = 0.0425
compounded_quarterly = year_interest_rate / 4
years = 0 #starting point
t = 0 #starting balance
#$30000 is the target
while t <= 30000:
    # Showing the number of quarterly periods, it need to divide by the number of q
    for q in range(4):
        t = t*(1 + compounded_quarterly) # Using one of equation to find out the nu
    t += invest
    years = years + 1
print(years, 'years') #printing the number of years to reach the target.
```

```
20 years
```

In [110…

```
print(f'Therefore, it will take {years} years to reach $30,000.')
```

```
Therefore, it will take 20 years to reach $30,000.
```